软件学报 ISSN 1000-9825, CODEN RUXUEW
Journal of Software, [doi: 10.13328/j.cnki.jos.000000]
©中国科学院软件研究所版权所有.
E-mail: jos@iscas.ac.cn
http://www.jos.org.cn
Tel: +86-10-62562563

面向航天控制系统的系统级需求描述语言*

陈小红[1*], 綦艳霞[2*], 阮锟[1], 林靖凯[1], 金芝[3,4+], 钟睿明[2], 顾斌[2+], 杨孟飞[5]

[1](华东师范大学 上海市高可信计算重点实验室, 上海 200062)

[2](北京控制工程研究所, 北京 100090)

[3](北京大学 高可信软件技术教育部重点实验室, 北京 100871)

[4](武汉大学 计算机学院, 武汉 430072)

[5](中国空间技术研究院, 北京 100094)

(*共同一作)

+通讯作者: 金芝, zhijin@pku.edu.cn,zhijin@whu.edu.cn 顾斌, gubinbj@sina.com

摘 要: 航天控制系统(ACS)构成复杂、软硬件交互紧密,其需求多采用自然语言描述,固有歧义性及硬件细节引用易出错,导致需求传递易生偏差,软件资产跨型号复用困难等问题.为解决这些问题,本文设计了面向航天控制系统的系统级需求描述语言 SpaceRDL. 其核心创新在于引入“系统资源”作为其交互环境实体与物理设备的统一抽象,使工程师能够以结构化方式精确定义对系统资源的操作,从而在需求描述语言层消除歧义性,避免引入过多细节. 该语言通过引入系统资源库,设计具有良定语义的原子操作指令,实现任务意图向软件实现的自动化映射,采用 SpaceRDL 描述的任务需求也成为跨项目迁移的可复用资产.案例研究表明,本语言能有效刻画航天控制系统需求,为构建新一代航天软件工程方法提供了切实可行的支撑.

关键词: 需求工程;航天控制系统;系统需求描述语言;需求规约

中图法分类号: TP311

中文引用格式: 陈小红,綦艳霞,阮锟,林靖凯,金芝,钟睿明,顾斌,杨孟飞. 面向航天控制系统的系统级需求描述语言.软件学报, 2021, 32(7). http://www.jos.org.cn/1000-9825/0000.htm

英文引用格式: Chen XH, Qi YX, Ruan K, Lin JK, Jin Z, Zhong RM, Gu B, Yang MF. A System-Level Requirements Description Language for Aerospace Control Systems. Ruan Jian Xue Bao/Journal of Software, 2021 (in Chinese). http://www.jos.org.cn/1000-9825/0000.htm

A System-Level Requirements Description Language for Aerospace Control Systems

Chen Xiao-Hong[1*], Qi Yan-Xia[2*], Ruan Kun[1], Lin Jing-Kai[1], Jin Zhi[3,4+], Zhong Rui-Ming[2], Gu Bin[2+], Yang Meng-Fei[5]

[1](Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai 200062, China)

[2](Beijing Institute of Control Engineering, Beijing 100094, China)

[3](Key Laboratory of High Confidence Software Technology (Peking University), Ministry of Education, Beijing 100871, China)

[4](School of Computer Science, Wuhan University, Wuhan 430072, China)

[5](China Academy of Space Technology, Beijing 100094, China)

Abstract: Aerospace control systems are characterized by high architectural complexity and tight software-hardware coupling. Their requirements are typically specified in natural language, which inherently suffers from ambiguity and is prone to errors when referencing low-level hardware details. These deficiencies often lead to inconsistencies during requirements propagation and hinder the reuse of software artefacts across projects. To address these issues, this paper proposes SpaceRDL, a system-level requirements description language specifically designed for aerospace control systems. Its core innovation lies in the introduction of “system resources” as a unified abstraction for the interactive environment entities and hardware devices, enabling engineers to define operations on system

* 基金项目: 国家自然科学基金(62192731, 62192730)

收稿时间: XXXX-XX-XX; 修改时间: XXXX-XX-XX; 采用时间: XXXX-XX-X

resources in a structured and precise manner, thereby eliminating ambiguity at the language level while avoiding excessive hardware-specific details. The language incorporates a system resource repository and features a set of atomic operation instructions with well-defined semantics, facilitating automated mapping from mission requirements to software realization requirements. Furthermore, requirements specified in SpaceRDL can be retained as reusable artefacts that support project migration. Case studies demonstrate that the proposed language effectively describes the requirements of aerospace control systems, providing a practical and viable foundation for the development of next-generation aerospace software engineering methodologies.

Key words: Requirements Engineering; Aerospace Control Systems (ACS); System Requirements Description Language; Requirements Specification

航天控制软件是空间飞行器的“大脑”与“神经中枢”,承担着姿态轨道控制、任务载荷管理、故障诊断与处置等关键功能[1].其行为的正确性与可靠性,是确保航天任务取得圆满成功的决定性因素,任何微小的需求偏差都可能导致灾难性的后果[2]. 因此,构建一套能够保证需求描述准确且可验证的开发体系,成为支撑我国航天事业持续创新与高质量发展的迫切需求[3].

目前大部分航天控制软件的开发都依赖以自然语言编写的软件需求规格说明,这成为航天控制软件开发自动化的瓶颈. 主要体现在两方面:其一,自然语言固有的模糊性与二义性, 使需求传递与理解产生歧义, 为软件的设计、实现与测试埋下错误隐患[4-6]; 其二, 航天控制软件需求描述中通常涉及特定硬件接口定义与设备底层特性[7-9],使核心软件资产难以在型号间复用或向新技术架构迁移,不利于领域经验与知识资产的系统化沉淀. 我们认为,要突破这一困境, 需要创建一种既能精确表达系统行为、又能独立于硬件细节的需求描述方法,即既需要提供足够高的抽象层级,从“系统”视角统一定义需求逻辑,屏蔽底层硬件细节;又需要具备足够精确的语义,以保证航天任务意图这一系统级需求向软件需求的正确转换.

现有的主流形式化方法难以同时兼顾这两方面要求. 典型的软件级需求建模描述语言[10-21]虽能通过精细的组件模型支持自动化转换, 但其建模视角始于软件架构, 缺乏对系统能力的统一抽象,无法在顶层有效锚定系统级需求;典型的系统级需求建模描述语言(如 SCR[22-24])虽能以“监视-控制”变量的形式刻画系统级功能关系,却在资源绑定的精确性、操作定义的原子性以及由此带来的需求资产复用性方面存在明显局限.

我们认为,需求工程过程中产生需求逻辑断层的根源在于“软件视角”与“系统视角”的分离.为此,本文提出了一种面向航天控制系统的系统级需求描述语言——SpaceRDL. 其核心创新是引入“系统资源”作为统一抽象层,将软件交互环境中的物理实体(如敏感器、执行机构)逻辑化为标准的、软件可定义的对象,同时构建基于系统资源库,并内嵌基于系统资源库的任务需求向软件实现需求的自动化映射机制.该机制支持工程师在控制逻辑层面,以结构化方式将任务意图解构为系统资源的行为效果序列,从而消除需求的歧义并屏蔽设备调用和底层特性,实现对航天控制软件需求的准确描述. SpaceRDL 的主要特性包括：

1)**需求描述的准确性**：系统需求(如“控制发动机点火”)被准确表达为对特定资源(如“点火指令寄存器”)在特定约束下的操作,使每一项需求均可追溯到具体的资源行为,并建立了系统需求、系统资源与软件需求间的映射关系.

2)**需求规约的可操作性**：基于上述映射关系,软件需求可以表达为对系统资源的原子操作及其逻辑组合,以及操作需满足的约束,使软件需求具备可执行性与可测试性.

3)**原子需求的可复用性**：通过可复用设备资源定义与需求模式,实现软件需求逻辑与硬件特性的分离.在设备升级或项目移植时,可独立进行资源实例化与绑定,不影响需求逻辑,提升软件的可迁移性.

本文的主要贡献包括：

- 提出“系统资源”的抽象,支持系统需求与交互环境的统一建模,建立航天控制系统资源库,为实现航天控制系统的需求开发自动化与高水平复用奠定基础.
- 设计航天控制系统需求描述语言 SpaceRDL,该语言以资源抽象为基本语言元素,结合航天业务逻辑特征,使工程师可聚焦于控制操作和控制逻辑的表达,其内置机制在资源库的支持下完成软件需求的自动化生成.

- 通过典型的航天器姿态控制案例,对 SpaceRDL 在工程实践中的可行性,以及其在提升需求复用性方面的有效性进行了实证研究.

本文后续结构安排如下：第 1 节分析航天控制系统的需求特征,并提出 SpaceRDL 的设计原则；第 2 节提出航天控制系统的资源抽象；第 3 节定义 SpaceRDL 语言；第 4 节介绍从 SpaceRDL 系统需求描述到软件需求规约的映射,并定义转换规则；第 5 节进行案例研究；第 6 节比较相关工作；最后, 第 7 节总结全文并展望未来研究方向.

1 航天控制系统需求特征及 SpaceRDL 设计原则

1.1 航天控制系统需求特征

航天控制系统通常涉及两个层次的需求：**系统需求**与**软件需求**.系统需求描述空间飞行器需要实现的高层任务意图,软件需求表达实现该任务意图要完成的具体操作以及操作过程中要满足的约束[25]. 基于经典需求工程框架[26,27],结合航天控制系统的特点,本文定位这两类需求的关系如图 1 所示.其中：

- **系统需求 SysR** 规定系统要“做什么”或者要完成的目标[25]. 在实践中,SysR 通常刻画环境实体或环境参数之间的一组约束,这些约束作用于物理设备(如敏感器、执行机构等)和数据存储等系统资源,可以是对功能或性能的约束.例如,“采集陀螺数据”和“系统控制周期为 128ms”.
- **软件需求 SofR** 则规定为了实现系统需求, 软件必须完成对系统资源的操作,以及操作所需要满足的约束,它定义软件“如何做”[25],即表达具体的、可执行的操作描述.例如, 为实现系统需求“采集陀螺数据”, 软件要“向陀螺发送取数指令”、“接收陀螺测量数据”、并“按照协议格式校验并处理陀螺测量数据”.

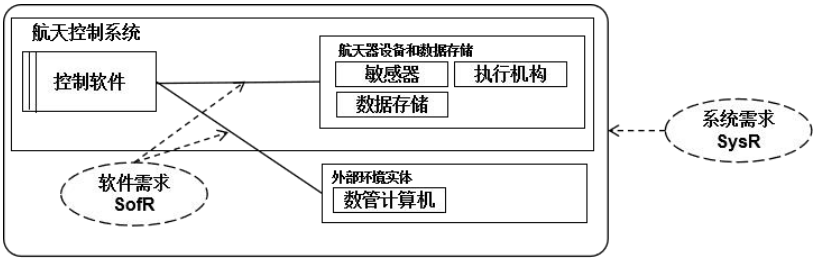


图 1 航天控制系统需求关系图

针对航天控制系统的需求工程,就是在给定空间飞行器系统需求 SysR 的前提下,根据空间飞行器中控制设备的能力,以及其它外部系统的约束,推断出软件需求 **SofR.**即给定软件的交互环境及其模型,根据系统需求 SysR,推导出软件需求 **SofR.**这一推导过程的本质依赖于对上述软件运行环境及其交互边界的精确刻画[28].

1.2 语言设计原则

为了确保航天控制系统的系统需求(任务意图)能够被精确描述,并能无歧义地推导出软件需求,本文制定了语言设计的如下四项原则：

- 原则一：面向资源的抽象原则

统一抽象软件交互对象为“系统资源”,并将其作为第一类建模对象.这使得系统需求能锚定在特定的系统组成成分或外部环境参数上,为软件需求的具像化推导提供基础. 具体体现为：提供具备独立语法构造的资源声明语句, 其中的资源标识符在整个推导过程保持唯一性和一致性.

- 原则二：需求描述的结构化原则

语言采用结构化语法结构,将需求组织为具有清晰层次与符合组合规则的规格结构.每个语法元素具有精确的语义解释,且通过结构化语法规则确保需求描述在传递过程中保持语义一致.

- 原则三：需求的可组合原则

语言支持从原子需求到复杂场景需求的层次化构建. 原子需求语句对应最基本的资源操作, 组合需求

语句通过结构化控制流模式将原子需求语句组织起来,形成完整的业务逻辑. 这种设计使该语言既可以表达细粒度的设备和外部系统访问操作, 也可以表达系统级的业务功能行为.

- 原则四：航天关键质量属性内嵌原则

语言将航天控制系统的关键质量属性引入需求规约,允许开发者在需求描述上附加时间(如任务周期、执行时限)、可靠性(如容错策略、冗余度)、安全性(如故障隔离、状态保持)等的非功能约束, 使其成为需求描述的内在组成成分,实现系统功能和非功能属性的绑定,保障系统功能与质量属性的同步满足.

2 航天控制系统资源抽象

2.1 资源抽象模型

航天控制系统中,控制软件通过与空间飞行器中的各类物理设备以及外部系统(统称为交互实体)间的交互来满足航天任务意图.其交互对象分为：一是物理实体,如敏感器（如陀螺和星敏感器）持续测量空间飞行器的姿态,执行机构（如推力器和动量轮）产生控制力矩;二是逻辑实体,比如姿态确定、轨道控制等算法,内含计算公式、控制流程与算法逻辑,分别描述输入输出的数学映射、模块间的执行顺序、以及内部的运算步骤;三是信息实体,比如系统中流转着陀螺角速度、滤波后姿态、控制力矩等各类代表物理环境状态的参数,其取值是决定控制算法和操作的条件.

本文将所有这些交互实体,统一抽象为可以明确定义的“系统资源”,并为资源封装和资源库构建提供统一框架.系统资源抽象遵循三个原则：

1) **功能与实现分离**：将交互实体能力与业务信息处理解耦,交互实体能力关注“能做什么”而非“为什么做”.
2) **数据格式标准化**：统一定义计算和传输的数据元素,建立无歧义的信息交换基础.
3) **独立功能模块化**：将独立控制算法和控制逻辑进行模块化封装,使其成为可验证、可复用的组件.

基于这些原则,本文构建了航天控制系统资源类型层次,如图 2 所示,其中：

- **物理设备**：对应物理实体,为航天控制系统中的硬件单元,包括控制计算机、系统设备(如敏感器和执行机构)和外部设备(如数管计算机).其中,控制计算机是负责核心计算与逻辑决策的硬件单元；系统设备是与航天任务核心功能直接相关的硬件单元；外部设备则位于系统边界,与系统存在交互,但其本身不属于控制核心功能范畴.
- **业务逻辑**：对应逻辑实体,分为三种：计算公式定义输入（如敏感器数据）到输出（如控制指令）的确定性数学映射关系,如姿态确定中的坐标变换；控制流程以流程图形式描述算法模块间的调用顺序、条件分支、循环迭代及异常处理路径,关注模块级的编排；算法逻辑以伪代码形式描述模块内部的具体实现步骤,关注模块内的运算与判断序列.
- **数据元素**：对应信息实体,包含航天控制系统涉及的所有数据元素的标准化定义,不仅定义数据元素的名称（即变量名）、数据类型和取值范围,还明确其物理意义.分为设备相关变量(如陀螺角速度、推力器指令等)和业务相关变量(如滤波后姿态、控制力矩等).前者直接来源于或作用于物理设备,反映硬件状态或向硬件下达指令；后者是经过算法处理后产生的中间或最终结果,用于支撑业务逻辑的决策与执行.

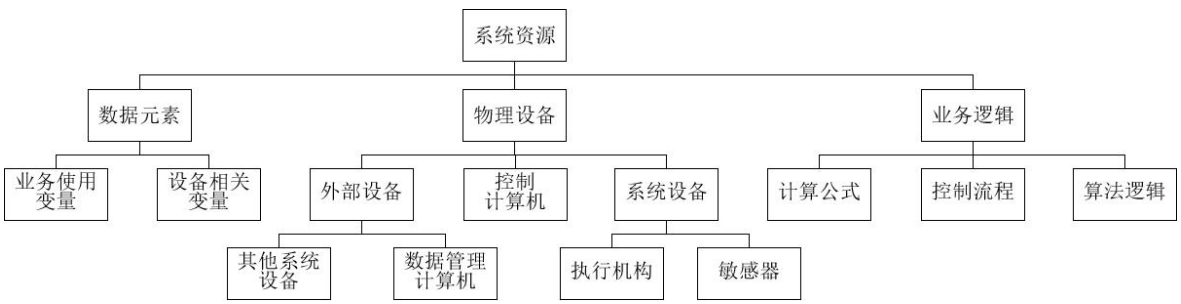


图 2 航天控制系统资源类型层次

2.2 资源库设计

基于上述系统资源类型层次,本文构建了四个资源库,提供结构化、可复用的资源描述,具体设计如下:

(1) 控制计算机库

控制计算机库存储控制计算机的硬件配置参数. 如表 1 所示, 其核心字段包括控制计算机名称、类型、CPU 型号, 定时器信息(包括 ID、时长、寄存器地址、配置说明、当量)、控制周期、中断信息(包括触发源、优先级等)、端口列表(包括端口类型、数量、ID、地址).

表 1 控制计算机库字段与含义

字段名称	含义描述
控制计算机 ID	控制计算机的唯一表示
控制计算机类型	控制计算机的类型
CPU 型号	处理器的型号
定时器信息	包含定时器 ID、时长、寄存器地址、配置说明、当量等信息
控制周期	系统控制周期
中断信息	包含中断触发源、优先级等配置
端口列表	包含端口类型、数量、ID、物理地址

(2) 系统设备库

系统设备库存储系统设备和外部设备的描述信息. 如表 2 所示, 其关键字段包括设备 ID、设备类型、设备数量、数据项(符号表示、物理含义、方向)、端口类型、通信协议和约束. 通过提供设备描述模板支持设备类型的复用和实例化,为设备更新和跨项目软件移植提供了基础.

表 2 系统设备库字段与含义

字段名称	含义描述
设备 ID	设备实例的唯一标识名称
设备类型	设备的分类(如陀螺、太阳敏感器、推力器等)
设备数量	同组设备的数量
数据项	包含符号表示、物理含义、数据方向(输入/输出)
端口类型	设备连接的物理端口类型
通信协议	设备通信的协议规范(如协议类型、数据帧格式等)
约束	设备的工作约束条件(如精度、量程、响应时间等)

(3) 算法库

算法公式库以模块化方式封装了系统的计算公式、控制流程和算法逻辑.如表 3 所示，条目结构包括算法 ID、功能名称、功能描述、输入变量、输出变量以及处理过程,其中“处理过程”字段以计算公式、流程图或伪代码描述业务逻辑.需求描述中,通过引用算法名称即可完整表达对应的业务逻辑。

表 3 算法库字段与含义

字段名称	含义描述
算法 ID	业务逻辑的唯一标识名称
功能名称	业务逻辑对应的功能名称
功能描述	功能的自然语言描述
输入变量	算法输入变量的列表
输出变量	算法输出变量的列表
处理过程	详细的处理过程, 可以用数学公式、伪代码或流程图表示

(4) 数据元素库

数据元素库标准化定义了系统中的所有环境物理量和控制变量. 如表 4 所示, 每个条目包含数据 ID、数据名称、数据类型、初始值、物理单位、有效范围、是否与设备交互、对应公式中的符号及其语义描述.

表 4 数据元素库字段与含义

字段名称	含义描述	字段名称	含义描述
数据 ID	变量的唯一标识符	物理单位	变量的物理单位(度/秒等)
数据名称	变量的语义化名称	有效范围	变量的有效取值范围
数据类型	变量的计算机内部表示(int、float 等)	是否与设备交互	1 表示与设备交互, 0 则反之
初始值	变量的初始值	物理单位	变量的物理单位(度/秒等)
公式中对应符号	变量在公式资源库中的表示形式	语义描述	变量含义和用途的详细说明

上述资源库通过资源 ID 相互关联,形成完整的资源描述体系,使系统需求、软件需求与交互对象三者能在能力维度上实现对齐,为实现软件需求的自动化推导与复用奠定基础.

2.3 资源库的构建与使用

建立系统资源抽象并提出统一资源建模和定义方法,目的是使资源信息可理解可检索.构建资源库则是为了管理航天控制系统资源实体,以支持系统资源能力描述的复用,为基于资源能力的软件需求规约奠定基础.

（1）资源库构建策略

资源库构建遵循“从具体型号中提炼,经资源间关联验证后固化”的原则.从航天飞行器型号研制开始,其物理设备、数据元素及业务逻辑将被抽象为资源条目.其中,控制计算机资源库从研制型号中提取 CPU 型号、定时器配置、控制周期、中断信息及端口列表等硬件基础环境参数;系统设备库基于设备描述模板,提炼数据项、端口类型、通信协议及工作约束等信息;外部设备按同样模板进行抽象,纳入系统设备库统一管理；数据元素库以软硬件接口为起点,按标准化格式录入设备相关变量与业务相关变量,优先复用已有条目,新增变量需经系统工程师与软件工程师联合评审;算法库从既有控制程序中提取可复用的算法单元,并将其输入输出变量与数据元素绑定.

所提炼出来的资源条目,需要通过关联验证方可入库.验证内容包括:设备资源的数据项符号与数据元素的定义保持一致,端口类型与通信协议相互匹配；算法公式中使用的输入/输出变量均在数据元素库中有明确定义,且数据类型与物理单位严格一致；控制计算机资源的定时器时长、中断优先级等参数,与设备资源的时序约束（如响应时间、数据更新频率）相容；设备资源引用的通信协议在算法库中有对应的解析逻辑,标记为“与设备交互”变量的数据元素至少要被一个物理设备引用;等等.资源库构建目前采用人工录入方式,未来将考虑引入基于大模型的辅助抽取方法,从领域文档中自动提取系统资源描述,构建系统资源库.

（2）资源库的检索与使用

为达成系统资源“一次定义、多次复用”的目标,本文设计了资源检索机制.控制计算机方面,需求描述中仅需指定计算机类型（如“AOCC”）,系统自动从资源库加载其预定义的定时器配置、中断信息、端口列表等完整配置,同一类型可在多个项目中直接复用.系统设备和外部设备方面,需求描述中仅需指定设备类型（如“Gyro_TypeA”）及实例名称,系统自动绑定资源库中预定义的数据项、端口类型、通信协议及约束条件,设备更换时仅需修改类型,需求主体结构不受影响.算法公式方面,需求描述中通过算法名（如“Kalman_Filter”）引用资源库中的算法公式,系统自动展开其输入输出变量与计算逻辑.数据元素方面,需求描述中直接使用变量名（如“角速度_测量值”）,系统自动从资源库获取其预定义的类型、单位、范围等信息.端口连接声明中只需指明控制计算机的端口类型与目标设备实例,端口参数及通信协议由双方资源库共同提供,系统可据此自动推导连接的一致性与兼容性.上述机制使工程师在撰写需求时无需重复定义资源细节，仅需引用资源 ID 或名称，即可完成对系统资源的完整描述与约束施加。

3 语言定义

本节设计面向航天控制系统的系统级需求描述语言——SpaceRDL.该语言以统一资源抽象为基础,构建标准化的系统需求描述模板,并定义原子操作指令和操作指令组合算子,其构成要素如下.

3.1 系统需求描述模板

航天控制系统的系统需求描述就是要定义空间飞行器的任务意图, 它通过有序的任务组织,描述需要完成的动作,以及任务执行时的一些基本约束. 主要包含三类任务:

- 初始化任务(InitTask):负责在系统上电或复位时建立系统的初始状态. 该任务是一次性执行的, 确保硬件接口和软件变量在进入周期性控制前处于预定的初始状态.
- 应用任务(AppTask):是系统的核心功能,该任务具有航天控制系统典型的周期控制特性,即按照预设的控制周期(如 128ms、1s 等)循环执行,完成敏感器数据的采集与处理、姿态确定与姿态控制计算、模式切换、执行机构控制指令输出以及遥控遥测处理等.
- 中断任务(InterruptTask):处理紧急异步事件的任务.该任务能够打断应用任务的正常执行,让系统对紧急事件进行即时响应.

这些复合型任务由基本的**原子操作语句**通过组合算子构建出来的结构化**组合语句**来描述.其中,原子操作语句直接对应不可再分的基础资源操作(如读写设备、调用算法、变量赋值),是需求描述的最小功能单元;而组合语句则由原子操作语句通过顺序、分支、循环等控制逻辑组合而成,能刻画完成特定任务意图的业务流程.

为了支撑系统需求的精确描述,SpaceRDL 定义了一个结构化系统需求描述模板,如图 3 所示.该模板包含四个部分:导入声明(Import)、系统组成(System)、任务描述(包括 InitTask/AppTask/InterruptTask) 与系统约束(SystemConstraints).这四个部分相互衔接, 构成从资源导入、系统构建、任务实现到约束确认的需求表达框架.

```
Import ControllerRepository;                    /*载入控制计算机库*/
Import DeviceRepository;                        /*载入系统设备库*/
Import AloForRepository;                        /*载入算法公式库*/
Import DataDictionary;                          /*载入数据元素库*/
System{系统组成语句（见3.2节）}                  /*系统组成*/
InitTask{任务描述语句见3.3节}                    /*初始化任务*/
AppTask {任务描述语句见3.3和3.4节}               /*应用任务任务*/
InterruptTask {任务描述语句见3.3和3.4节}         /*中断任务*/
SystemConstraints{系统约束语句见3.5节}           /*整体系统的约束 */
```

图 3 航天控制系统需求描述模板

3.2 导入声明和系统组成

导入声明是 SpaceRDL 语言的基础组成部分,通过导入第 2 节构建的系统资源库,获得标准化的领域术语和概念关系,为系统需求描述提供控制逻辑描述的词汇表和上下文.同时也支持资源定义的复用和标准化, 确保不同类型航天项目中资源描述的一致性. 导入声明的描述形式如下:

Import repoName

其中, repoName 为资源库名称, 包括 DeviceRepository(系统设备库)、ControllerRepository(控制计算机库)、AlgorithmRepository(算法库)或 DataDictionary(数据元素库)等.

系统组成声明描述了航天控制系统的硬件架构,不仅包括控制计算机(**Controller**)、系统设备(**SystemDevice**)、外部设备(ExternalDevice)等物理设备,还包括设备之间的端口连接(**Connect**). 该声明刻画了系统的硬件拓扑结构与设备间的互联关系,为接口调用与时序约束验证提供依据.其中控制计算机代表该空间飞行器的控制计算机,系统设备和外部设备（统称交互设备）代表与控制计算机交互的各个物理设备. 由于系统资源库中预定义了控制计算机和系统设备、外部设备等信息, SpaceRDL 中只需声明相应的类型、名称和具体的控制周期.

端口连接用于显式表达控制计算机端口与交互设备端口之间的连接关系.由于系统资源库中包含了各个

交互设备的类型(每个类型对应的数据接收和发送的等待时间、内容及格式是确定的)和控制计算机的类型(每个类型对应的各个接口的类型及配置参数是确定的). 端口连接声明中只需要引用对应交互设备的名称, 以及直接引用对应控制计算机类型的接口类型即可. 它们的描述形式如下:

- **控制计算机声明**: **Controller** SubType ctlName{ **ControlPeriod**: periodNum}

其中, SubType 指定控制计算机的具体类型(如 AOCC 姿态轨道控制计算机), ctlName 为实例名称, periodNum 为系统的控制周期.

- **交互设备声明**: **Device** SubType deviceName

其中, SubType 指定交互设备的具体类型（含系统设备与外部设备）, deviceName 为设备名称.

- **端口连接声明**: **Connect** PortID deviceName

其中, PortID 指定具体端口类型, deviceName 为交互设备名称

3.3 原子操作语句

原子操作语句是 SpaceRDL 中最基本的需求描述单元,对应于航天控制系统中的不可再分的功能操作,本小节首先设计操作指令,并引入对操作的时间约束.

(1) 操作指令设计

航天控制系统包含姿态、轨道、热控、能源及数传等功能域.在软件需求层面,各功能域的动作类型边界清晰、枚举空间有限且相对稳定,具备抽象为标准化原子操作的前提.但控制操作在轨执行时所涉及的目标设备、控制阈值、时间约束及数据源地址等,需由当前轨道动力学状态、能源可用条件、设备健康情况及故障处置策略等实时决定,并在生命周期内持续变化.这导致指令设置上的困难,即动作类型固定而执行数据动态变化.若将两者绑定于同一语法实体,数据的动态变更将波及动作类型定义,导致指令体系脆弱且很难维护.本文采用结构化解耦策略来设计操作指令,即显式划分操作指令算子与操作参数.指令算子表征动作类型,界定系统响应的行为类别;操作参数承载该动作类型的执行数据,描述操作目标、数值范围及执行约束.指令算子保持跨任务的逻辑稳定性,操作参数则支持独立更新与动态重构,从而支持操作指令的可复用、可配置及可验证.

操作指令算子与操作参数遵循不同的设计原则.操作指令算子的定义依赖于对航天控制软件功能需求的分析.本文将姿态控制、热控调度、数据下传等航天领域典型任务按问题模式[29]进行分类,根据问题模式的控制动作提取操作语义,形成标准化操作指令算子集.操作参数则根据具体控制操作执行上下文的解析来获得,其维度与取值随任务配置与运行状态动态变化.

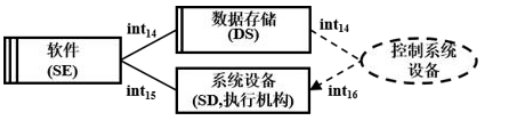
首先,依据第 2.1 节的系统资源分类,将操作参与者划分为系统设备、外部设备及数据存储三类.其中,系统设备与外部设备对应物理设备分类,是软件在物理层面的操作对象; 数据存储对应业务逻辑中负责数据持久化管理的部分,是软件在信息存储层面的操作对象.参照文献[29]的问题模式,对上述三类参与者进行分类组合,识别出 6 种原子操作模式,如表 5 所示.其中,初始化模式对系统设备与数据存储执行初始化操作;数据采集模式用于获取目标数据;控制计算模式实现输入数据至输出数据的转换;控制输出模式对系统设备施加控制作用;遥控处理模式接收来自外部设备的数据;遥测处理模式将遥测数据发送至外部设备.这些模式被定义为操作指令算子,分别定义为 **Init**、**Set**(硬件和软件初始化/置位), **GetData**(数据采集), **Compute**(控制计算), **SendData**(控制输出), **ProTC**(遥控处理)及 **ProTM**(遥测处理).

在此基础上,我们再根据原子操作模式设计算子操作参数.针对任一原子操作模式,根据各参与者在模式中承担的语义角色(数据源,数据目标,控制对象或通信端点),为其分配相应的参数占位符,将其组合为操作指令的形式化参数列表.以数据采集模式为例,该模式涉及系统设备与数据存储两类资源,其中系统设备充当数据源角色,数据存储充当数据目标角色,因此提取 deviceName 与 targetData 两个参数,构成操作指令“**GetData** deviceName targetData”;其中 deviceName 指代待读取的系统设备实例,targetData 指代待采集的目标数据.其余五种原子操作模式的参数遵循同一推导流程获得,具体定义如表 5 所示.

(2) 原子操作语句时间约束

在确定好操作指令后还要引入时间约束. 基于航天控制系统的时序需求, 原子操作语句的时间约束需覆

表 5 航天控制系统原子操作模式

原子操作模式	模式解释及对应操作指令
 int1:<SE, SD, iniConfig>[Value] int3:<SD, SE, config>[State] int2:<SE, DS, IniValue>[Value] int4:<DS, SE, value>[Value]	模式名称：初始化 任务意图：对系统设备和数据存储进行初始化 软件需求：软件向系统设备写入初始配置， 软件向数据存储写入变量初始值。 操作指令： 初始化软件配置：**Set** variabelName value 初始化硬件配置：**Init** portName config
 int8:<SD, SE, rawData>[Value/State] int9:<SE, DS, writeOutput>[Event] int10:<DT, SE, targetData>[Value]	模式名称：数据采集 任务意图：从系统设备采集目标数据 软件需求：软件从系统设备中读入原始数据， 软件向数据存储中存入目标数据。 操作指令： **GetData** deviceName targetData
 int5:<DS, SE, inputData>[Value] int7:<DT, SE, outputData>[Value] int6: <SE, DT, writeOutput>[Event]	模式名称：控制计算 任务意图：将输入数据转换为输出输出 软件需求：软件从数据存储1读取输入数据， 软件将输出结果写入数据存储2。 操作指令： **Compute** outputData inputData
 int14:<DS, SE, controlInstruction>[Event] int15:<SE,SD, controlSignal>[Event] int16:<SD,SE, deviceState>[state]	模式名称：控制输出 任务意图：对系统设备进行控制，使其处于某种状态 软件需求：软件从数据存储获取控制指令， 软件发送控制信号至系统设备 操作指令： **SendData** deviceName deviceState
 int11:<ED, SE, data>[Value] int13:<DS, SE, parsedData >[Value] int12:<SE, DS, parseData>[Event]	模式名称：遥控处理 任务意图：从外部设备接收数据 软件需求：软件从外部设备接收数据， 软件将完成协议解析后的数据写入数据存储。 操作指令： **ProTC** externalDeviceName parsedData
 int17:<DS, SE, processData>[value] int18:<SE, ED, packData>[Event] int19:<ED, SE, PackedData>[value]	模式名称：遥测处理 任务意图：将遥测数据发送给外部设备 软件需求：软件从数据存储读取遥测数据， 软件按协议打包后发送至指定外部设备obdh。 操作指令： **ProTM** externalDeviceName packedData

盖三类问题：一是单个任务运行时长的管控，避免占用空间飞行器有限计算资源影响后续周期任务；二是任务的精确时刻触发，满足任务在绝对时刻或周期内时刻触发的需求；三是时间窗口内的任务执行频次，服务于系统可靠性验证或事件统计场景.

对应地,SpaceRDL 提供三类时间约束：**运行时间约束、指定时刻发生约束和累积时间发生约束**. 其中，运行时间约束限定任务的最长执行时长；指定时刻发生约束分为绝对时间与周期内时间两种,为任务触发提供时刻基准；累积时间发生约束用于刻画特定时间窗口内的执行次数,且区分“连续执行”与“非连续执行”. SpaceRDL 分别给出这三种约束的表达形式,具体含义及其语法如表 6 所示. 其中,F 为原子操作,t、t1、t2 均为时刻,T 为时间段,n 为次数.

(3) 原子操作语句表示

原子操作语句可表示为：

[调用时间约束] 原子**操作** [运行时间约束]

表 6 SpaceRDL 原子操作的时间约束

约束类型		功能描述	语法形式
运行时间约束		确保 F 在规定时间 T 内完成.	F **Finished Within** T
调用时间约束	指定时刻发生约束	从绝对时间 0 时刻算起的 t 时刻执行 F	**At** (t) F
		从周期内 0 时刻算起的 t 时刻执行 F	**At**(t, **period**) F
	累积时间发生约束	T 时间内, 连续发生 n 次 F	**Over**(T, **c**, n) F
		T 时间内, 非连续发生 n 次 F	**Over**(T, **nc**, n) F

其中, 调用时间约束、运行时间约束均为可选项,原子操作中除操作指令之外,还包含 ReqCapByFormTable formDesc 字段, 表示根据系统资源库算法公式表中定义的算法完成该操作.

SpaceRDL 定义了如表 7 所示的原子操作语句模板.只有 Compute 类算子的参数中包含 ReqCapByFormTable formDesc 必选项,因为 Compute 涉及数值计算,其转换逻辑须精确关联公式资源库中的算法公式定义.无法仅凭算子名称与简单参数准确定义其行为,必须借助外部形式化描述（即 formDesc）来承载完整的执行规约.

表 7 SpaceRDL 原子操作语句模板

语句模板类	操作算子	语句模板	语句操作含义
设备相关类语句	**Init**	[调用时间约束] **Init** portName config [**Finished Within** T]	<在调用时间约束下>,<在 T 时间内>完成将 portName 标识的硬件端口按照 config 配置结构体进行初始化配置
	GetData	[调用时间约束] **GetData** devName devdataName [**Finished Within** T]	<在调用时间约束下>,<在 T 时间内>完成从 devName 标识的设备采集数据 devDataName
	SendData	[调用时间约束] **SendData** devName devdataName [**Finished Within** T]	<在调用时间约束下>,<在 T 时间内>向 devName 标识的设备发送控制信号 devDataName
	ProTM	[调用时间约束] **ProTM** obdhName dataName [**Finished Within** T]	<在调用时间约束下>,<在 T 时间内>完成按照 obdbName 标识数据管理计算机的通信协议将数据 dataName 进行打包, 并将打包后数据发送给 obdhName 标识的数据管理计算机.
	ProTC	[调用时间约束] **ProTC** obdhName dataName [**Finished Within** T]	<在调用时间约束下>,<在 T 时间内>完成从 obdhName 标识的数据管理计算机接收遥控指令, 并按照 obdbName 标识的数据管理计算机通信协议将遥控指令解析为遥控数据 dataName.
数据处理类语句	**Set**	[调用时间约束] **Set** varName value [**Finished Within** T]	<在调用时间约束下>,<在 T 时间内>完成将 varName 标识的变量赋值为 value 标识的数值.
	Compute	[调用时间约束] **Compute** outputdata inputdata **ReqCapByFormTable** formDesc [**Finished Within** T]	<在调用时间约束下>,<在 T 时间内>完成按照 formDesc 标识的算法公式将源数据 inputdata 转化为目标数据 outputdata

3.4 组合算子及组合操作语句

SpaceRDL 定义了操作语句的组合机制,来支持复杂任务流程的描述.它通过结构化控制流模式(如顺序、分支、循环等),将操作语句组织起来形成任务逻辑单元.SpaceRDL 提供四类共 5 个组合算子,如表 8 所示.这些组合算子可嵌套使用其中,F1、F2 为原子操作语句或者组合操作语句.顺序组合语句中可以规定其中任意两个语句之间的间隔时间.

3.5 系统约束

系统约束主要包括四类：时间约束,以 **Finished Within** T 语法形式限定系统整体的执行截止期限；空间约束,以 Executed Within M 语法形式对内存占用量施加上限（如 **Executed Within** 32kb）；编译工具与平台约束,以 **Compile With** Options 语法形式明确目标平台、编译工具链及优化选项; 编程语言约束,以 Language Standard 语法形式限定源语言的版本及编码规范(如 **Language Standard** C11).

表 8 SpaceRDL 组合算子及组合操作语句

类型	组合算子	语句模板	含义
顺序	**Composite**(不发生歧义的情况下,仅用“;”作为间隔符)	**Composite**{F1; [**In**(T1) | **In**[t1, t2] | **After**(T1)] F2; ...} [**Finished Within** T]	组合操作先执行 F1,再执行 F2,总执行时长在 T 时间内.其中的间隔时间约束含义如下: (1) In(T1)表示 F1 发生后, 等待 T1 时间后执行 F2, 且该时间段内不可以执行其他操作 (2) In[t1, t2]表示F1发生后, 等待t1-t2时间段后执行F2, 且该时间段内不可以执行其他操作 (3) After(T1)表示 F1 与 F2 发生的间隔在 T1 时间内
选择	**If Else**	**If**(condExpr){F1}[**Else**{F2}][**Finished Within** T]	如果条件 condExpr 满足, 则执行 F1, 否则执行 F2, 执行时长要在 T 时间内
	Switch Case	**Switch**(varName){**Case**(value): {F1};...}[**Finished Within** T]	根据匹配变量 varName 的值, 选择执行: 若等于 value, 则执行 F1,执行时长要在 T 时间内
循环	**While**	**While**(condExpr){F1}[**Finished Within** T]	如条件 condExpr 满足则循环执行 F1,执行时长要在 T 时间内;若条件中包含时间,则按照该时间违反执行循环跳出
并发	**Parallel**	**Parallel**{F1 || F2||...} [**Finished Within** T]	F1 和 F2 并发执行,执行时长要在 T 时间内

3.6 SpaceRDL总结

如表 9 所示,SpaceRDL 共定义了 46 个保留字, 涵盖三个类别.其中,功能操作类包括原子操作算子共 11 个, 逻辑表达类包括组合算子与调用时间约束关键字, 共 12 个, 需求描述类包含资源声明等关键字共 23 个. SpaceRDL 提供了 7 类原子操作算子(可形成 396 条原子操作语句类型)和 4 类 5 个组合算子(可形成 13 条组合语句类型) .

表 9 关键字类型与操作语句类型统计表

类型		数量
关键词类型	功能操作类	11
	逻辑表达类	12
	需求描述类	23
操作语句类型	原子语句类	396
	组合语句类	13

4 从 SpaceRDL 需求描述到软件需求的映射规则

本节阐述如何将 SpaceRDL 规约转换为指导设计与实现的软件需求.转换过程遵循结构化方法,

4.1 原子操作语句映射

原子操作语句映射以表 5 所列原子操作模式为依据.依据语句功能特性,可将其划分为设备相关类与数据处理类.设备相关类语句需处理硬件端口、通信协议等设备信号级底层细节, 而数据处理类语句需要引入算法库所提供的业务逻辑. 分别建立对应的软件需求映射规则,如表 10 所示.

- **设备相关类**: 根据目标设备从资源库中获取硬件信息, 并确定软件与设备之间的交互数据;获取每个交互数据关联的端口和通信协议;获取软件与设备在交互时应遵循物理约束,包括设备响应时间、数据发送间隔等;按照各类语句的模板组织上述信息.
- **数据处理类**: 根据操作算子识别功能类型,从资源库中获取该数据处理功能使用的算法公式;获取数据处理时应遵循的约束,如时间约束等;按照各类语句的模板组织上述信息.

4.2 组合语句映射规则

根据表 8,我们将其中由顺序、选择、循环等组合算子描述的复合任务逻辑,映射为软件需要进行的控制流结构,其对应关系如表 11 所示,其中 S1 和 S2 为表 8 中 F1 和 F2 对应的软件需求. 映射依据在于组合算子与控

表 10 原子操作语句映射规则及软件需求模板

模板类型	操作算子	对应软件需求模板
设备相关类	Init	在<调用时间约束>条件下, 软件将<目标设备>的<硬件端口>进行初始化, 配置参数为<具体参数>, 以上执行时间应在<运行时间约束>内.
	GetData	在<调用时间约束>条件下, 软件通过<硬件端口>向<目标设备>发送<操作指令>, 格式遵循<通信协议>, 字节发送间隔小于<时序要求>, <目标设备>接收<操作指令>后在<响应时间>内完成<指定操作>; 软件通过<硬件端口>接收<目标设备>发送的<数据>, 根据<数据格式>验证<校验项>, 若通过验证, 则<执行相应操作>; 以上执行时间应在<运行时间约束>内.
	SendData	在<调用时间约束>条件下, 软件通过<硬件端口>向<目标设备>发送<操作指令>, 数据格式遵循<通信协议>, 字节发送间隔小于<时序要求>, <目标设备>接收<操作指令>后在<响应时间>内完成<指定操作>, 以上执行时间应在<运行时间约束>内.
	ProTM	在<调用时间约束>条件下, 软件通过<硬件端口>向<目标设备>发送<操作指令>, 数据格式遵循<通信协议>, 字节发送间隔小于<时序要求>, 以上执行时间应在<运行时间约束>内.
	ProTC	在<调用时间约束>条件下, 软件通过<硬件端口>接收<目标设备>发送的<数据>, 根据<通信协议>验证<校验项>, 若通过验证, 则<执行相应操作>, 以上执行时间应在<运行时间约束>内.
数据处理类	Set	在<调用时间约束>条件下, 软件将<目标对象>赋值为<目标值>, 以上执行时间应在<运行时间约束>内.
	Compute	在<调用时间约束>条件下, 软件将<输入数据>转换为<输出数据>, 使用<算法公式>进行<功能操作>, 计算精度满足<性能指标>, 以上执行时间应在<运行时间约束>内.

制流结构在执行语义上的等价性.映射以原子操作语句对应的软件功能 S 为组成单元,以运行时间约束为时序验证条件,以保证转换后控制流与原始任务逻辑执行行为等价且满足时间约束.

表 11 组合语句映射规则及软件需求模版

语句类型	组合算子	组合语句对应的软件需求模板
顺序	Composite	软件按照严格的先后顺序依次执行：S1,S2. 组合执行时间不超过<运行时间约束>. S1,S2 中间若有间隔时间,按照间隔时间执行
选择	If Else	软件根据<condExpr>的布尔结果选择分支执行. 若满足则执行：<S1>, 否则执行：<S2>. 各分支路径的执行时间不超过<运行时间约束>.
	Switch Case	软件根据<condExpr>的结果选择分支执行. 若结果为<数值 1>, 则执行：<S1>, 若结果为<数值 2>, 则执行：<S2>. 各分支路径的执行时间不超过<运行时间约束>.
循环	While	软件在<循环条件>满足时重复执行：<S>；单次迭代时间不超过<运行时间约束>.
并发	Parallel	软件并发执行：S1,S2. 整体并发执行时间满足<运行时间约束>.

5 案例研究

本节以太阳搜索控制系统为案例,验证 SpaceRDL 在典型航天多模式控制系统需求描述中的表达能力与应用效果. 该系统的主要功能是实现太阳捕获:通过采集陀螺与太阳敏感器的测量数据确定卫星当前姿态, 控制卫星绕俯仰或滚动轴转动, 使太阳进入敏感器视场并最终维持对日定向姿态. 太阳搜索控制系统由太阳敏感器、陀螺、推力器与控制计算机组成. 系统通过陀螺和太阳敏感器测量姿态信息,利用推力器执行控制指令, 实现多模式的姿态调整和稳定控制.本章节选取该需求中具有代表性的部分进行阐述,完整的 SpaceRDL 描述及对应的软件需求规格说明详见 http://re4cps.org/spacerdl-appendix/. 需要说明的是,在进行本案例之前,已经把案例里涉及的系统资源录入了系统资源库中,可以直接使用.

5.1 导入声明与系统组成运行平台声明

在 SpaceRDL 中, 首先需要通过资源声明语句导入系统运行所依赖的外部资源. 通过 import 语句, 可将相应的资源库引入到模型中. 在本案例中, 系统需导入系统设备库(DeviceRepository)、控制计算机库(ControllerRepository)、算法库(AlgorithmRepository)以及数据元素库(DataDictionary).在太阳搜索系统中,控制系统组成包括一个控制计算机 aocc，4 个太阳敏感器、9 个陀螺、12 个推力器以及 1 个外部数据管理计算机,其中太阳敏感器和陀螺属于敏感器,推力器属于执行机构.在太阳搜索系统中, 控制计算机的控制周期为 128ms,其对应的 SpaceRDL 语句为：**Controller** Aocc aocc { **ControlPeriod**: 128ms }; 每个(组)设备都可以通过

名字唯一标识,比如 **Device** Gyro3F gyro1. 其中, Gyro3F 表示设备类型为三浮陀螺, 而 gyro1 为该设备的名称,其数量为 9,表示 9 个陀螺. 其他设备可以据此进行定义.

此外, 还需要指明每个设备与控制计算机之间的连接,以建立正确的设备绑定关系.每个设备可能由多个端口与控制计算机相连,比如,**Connect** { UARTport5, UARTport4 } obdh1 就是将 aocc 的 UARTport5 和 UARTport4 两个端口与数据管理计算机 obdh1 连接,这两个端口在控制计算机库中已经明确定义.本案例中,其他设备都是通过一个端口与 aocc 相连.

5.2 任务与约束

任务定义模块包括了初始化任务、应用任务和中断任务三类. 初始化任务负责系统在上电后的关键初始化操作, 如通信端口配置、设备加电以及定时器启动. 本案例中, 初始化任务主要完成所有外设通信端口的初始化. 以陀螺通信端口 UARTport1 为例,需设置停止位、校验位及波特率,对应 SpaceRDL 语句为: **Init** UARTport1 {0, 0, 250000}; 分别将停止位与校验位设为 0,波特率设为 250000.软件变量全部初始化为 0.

对于周期控制任务的控制周期为 128 ms,而需求规定其完成时间不超过 100 ms. 在 SpaceRDL 中通过 **AppTask**{} 语句的末尾的“**Finished Within** 100 ms”进行声明.该周期内依次完成遥控处理、陀螺数据采集、太阳敏感器数据采集、陀螺数据剔野、陀螺姿态确定、陀螺与太阳敏感器姿态确定、模式处理、控制量计算以及遥测处理共 9 项功能. 其中,模式处理涉及多条件判断与分支决策,逻辑关系较为复杂,需采用组合操作语句进行描述;其余任务均为原子功能,通过原子操作语句描述.

原子操作语句的书写方式如下,对于每个原子功能,首先根据其功能类型确定对应的操作算子.以本案例中的"陀螺数据采集"功能为例,其功能类型对应 **GetData** 算子.随后,需确定该算子的操作参数,包括目标设备名称及待采集的数据项.该功能中,设备名称为 gyro1,输出数据包括“加电状态”(GyroPowerState)与“角速度模拟量”(deltG),两者在数据元素库中的英文标识分别为 deltG 和 GyroPowerState.据此, 该功能的 SpaceRDL 语句定义如下: **GetData** gyro1 {deltG[9], GyroPowerState[9]},方括号内的数字 9 表示分别采集 9 个陀螺的测量结果.其余原子功能的定义方式依此类推.

组合操作语句的构建遵循以下方法:首先,分析复杂任务的控制逻辑,将其拆解为若干顺序执行的阶段,并识别各阶段内的条件判断与分支决策点;其次,将各阶段中不可再分的功能单元识别为原子操作,确定其对应的算子类型及操作参数;然后,依据各阶段内的控制流特征选择合适的组合算子——顺序组合描述串行流程,条件分支描述决策路径, 循环结构描述迭代行为;最后,按照任务执行时序,将原子语句以组合算子为骨架进行层次化嵌套.以本案例中的“模式处理”需求为例,该需求刻画了控制软件在速率阻尼(RDSM)、俯仰搜索(PASM)、滚动搜索(RASM)和对日巡航(CSM)四种工作模式之间的自动切换行为,其逻辑过程包含多条件判断与分支决策,关系复杂,难以通过单条原子语句直接表达.通过对该需求进行分析与分解,其处理流程包含三个阶段:

首先, 系统根据当前状态数据计算下一控制周期的工作模式; 随后, 判断下一周期工作模式与本周期工作模式是否一致. 若两者相同, 则对下一周期模式执行初始化操作;最后, 根据确定的下一周期工作模式, 对三轴目标角速度进行设置. 在 SpaceRDL 中,该需求通过以下原子语句与组合逻辑实现完整描述:由 **Compute** 原子语句依据算法库 B5 条目(下一个模式字计算公式)计算下一周期模式; 通过 **If-Else** 组合语句判断模式一致性, 并在条件满足时调用 **Set** 原子语句执行初始化;通过 **Switch-Case** 组合语句构建多模式分支结构,在各分支中调用 **Set** 原子语句设置三轴目标角速度.通过上述结构,复杂的模式切换逻辑被多条原子语句及其组合关系有效刻画,完整展现了系统的控制过程,其对应的 SpaceRDL 代码实现如下所示.

```
Composite modeProcess{
    Compute {nextMode} {currentMode, viewCode, cntMode, attirate} ReqCapByTable FormulaTable.B5;
    If (currentMode!= nextMode){
        Set currentMode nextMode;
        Set cntMode 0;
    }
    Switch (nextmode) {
        Case(RDSM): Set attirateT {0.0, 0.0, 0.0};
        Case(PASM): Set attirateT {0.0, 0.5, 0.0};
```

```
            Case(RASM): Set attirateT {0.5, 0.0, 0.0};
            Case(CSM): Set attirateT {0.0, 0.0, 0.0};
        }
}
```

中断任务本质为由时间触发条件与原子操作构成的组合结构,遵循组合语句的统一构建流程. 本案例的中断任务要求在每个控制周期的 110ms 时刻触发, 并在 100 us 内完成脉宽指令输出. 其原子操作对应 **SendData** 算子, 交互对象为设备实例 jet1, 输出数据为 pulseWidth. 据此,生成的 SpaceRDL 语句如下:**At**(110ms, **period**) **SendData** jet1 pulseWidth **Finished Within** 100us.

最后,需考虑系统约束,用于描述航天控制系统在时间与空间维度上的全局性限制条件.本案例中, 系统存在一项空间资源约束:应用软件编译后的二进制文件(bin 文件)大小不得超过 32 KB. 该约束在 SpaceRDL 中可通过以下语句进行描述: **Executed Within** 32 kb; 该语句明确限定了系统的存储空间上限, 为后续的软件实现与资源优化提供了约束依据.

5.3 太阳搜索系统的软件需求规约

根据第 4 章中提出的系统需求到软件需求的映射规则, 可以实现系统需求向软件需求的自动化映射.具体的转换结果见 http://re4cps.org/spacerdl-appendix/.下面分别以原子操作语句和组合操作语句为例,说明映射的结果. 对于原子操作语句“**At**(5ms, **period**) **GetData** gyro1 {deltG[9], GyroPowerState[9]}”,其映射过程如下. 首先, 从系统资源库中获取目标设备 gyro1 的硬件信息, 并确定软件与设备之间的交互数据为取数指令和陀螺测量数据{deltG[9], GyroPowerState[9]}; 然后查询到软件与 gyro1 交互取数指令和测量数据需要经过异步串口 UARTport1 以及取数指令协议表和陀螺测量数据格式表. 最后根据表 10 所示的模板组织这些信息, 其转换后的软件需求描述为“在每个控制周期开始后的第 5ms, 软件通过异步串口 UARTport1 向 gyro1 发送两字节取数指令 0xEB91, gyro1 接收取数指令后在 3ms 内完成测量数据的采集并发送给软件; 软件通过异步串口 UARTport1 接收 gyro1 发送的测量数据{deltG[9],GyroPowerState[9]}, 根据陀螺测量数据格式表,校验数据长度、帧头和累加和,若通过验证,则存储数据.

对于组合操作语句“**If**(currentmode!=nextmode){**Set** currentmode nextmode; **Set** cntMode 0;}**Finished Within** 1ms”其转换后的软件需求描述为“软件根据 currentmode != nextmode 的布尔结果选择分支执行. 若满足则执行: 软件按照严格的先后顺序依次执行: 1).软件将 nextmode 的值赋值给 currentmode. 2).软件将 0 赋值给 cntMode. 各分支路径的执行时间不超过 1ms. ”

5.4 工具支持与应用效果

为支持 SpaceRDL 所设计实现的原型工具,部署于在线平台(网址: http://re4cps.org:7080/spacerdl-editor/). 该工具提供需求编辑、静态检查、自动补全及软件需求规约生成四项功能.其中,需求编辑环节以人工参与为主,用户通过页面进行 SpaceRDL 描述的编辑,工具提供语法高亮、结构提示等辅助能力; 其余三个环节均由工具自动完成: 静态检查负责验证需求的类型一致性、资源引用有效性及变量定义完整性; 自动补全基于资源本体与语法规则,对不完整的需求描述进行参数和结构补全; 软件需求规约生成则将最终的需求描述转换为可供软件设计使用的规约文档.

为考察 SpaceRDL 在航天控制系统需求描述中的应用效果,除太阳搜索控制系统外,还选取对地定向、对日定向两个典型案例,在工具支持下手动完成分析.参与应用的专家具有 20 余年航天控制系统研发经验,长期从事航天控制软件的需求分析与设计工作.

三个案例的应用情况如下:太阳搜索控制系统的自然语言需求文档为 70 页,用 SpaceRDL 描述仅需 55 行结构化语句;对地定向和对日定向的自然语言需求文档均在 100 页左右,用 SpaceRDL 描述分别为 87 行和 112 行. 太阳搜索、对地定向、对日定向的自然语言需求文档分别存在 18 处、27 处、32 处条件模糊、边界不清或指代不明的歧义点,在 SpaceRDL 描述中全部消除,未引入新的歧义项.在对地定向与对日定向案例中,喷管喷气脉宽设置、岁差章动计算、姿态控制律计算等多个功能模块,共 19 条 SpaceRDL 语句可直接复用或经少量参数调整后复用,约占周期控制需求描述总量的 70%.

6 相关工作

与此相关的工作涉及面向嵌入式系统的需求描述语言,分为嵌入式系统软件需求描述语言与系统需求描述方法.

6.1 嵌入式系统软件需求描述语言

根据建模关注点,嵌入式系统软件需求描述语言可分为两类:

行为响应类:以 Statecharts [10]、Stateflow [11] 和 SDL [12]为代表,侧重于描述系统对外部事件、信号或消息的响应逻辑与状态迁移规律,适用于表达反应式系统与通信协议需求模型. Statecharts 通过引入层次化状态、并发区域与广播通信机制,增强了传统有限状态机的表达能力,主要用于描述复杂反应式系统需求模型. Stateflow 在 Statecharts 基础上进一步整合流程图与事件逻辑,支持嵌入式控制逻辑可视化. SDL 则侧重于刻画通信协议与进程交互模型,适用于消息驱动的嵌入式系统.

时序模式类:如 Modechart [13]、SPARDL [14] 和 Giotto [15], 强调系统的运行模式、周期任务及时间触发机制, 支持多模式切换与实时调度描述. Modechart 支持图形化建模, 适用于表达具有多模式切换的实时系统. SPARDL 结合模式转换与周期任务调度, 支持控制系统中复杂任务流程描述. Giotto 则通过时间触发模型实现任务逻辑与调度机制的分离.与时序模式相关的工作, 还包括 FRORL[16,17]、RT-FRORL[18]与 ASLAN[19]、RT-ASLAN[20]、ASTRAL [21]等, 它们基于时序逻辑与断言描述系统行为.

上述需求描述语言普遍聚焦于软件行为层面——行为响应类关注状态迁移逻辑,时序模式类关注任务调度与模式切换——但均未涉及系统能力需求与物理设备抽象.以航天领域的 SPARDL 为例,其虽在模式转换与周期任务时序行为方面描述精细,但建模视角仍局限于软件,需求描述与具体设备变量紧密绑定,缺乏统一的物理资源抽象机制,难以支持跨型号复用.

6.2 系统需求描述方法

目前涉及系统需求描述的方法包括:

基于模板与受限自然语言的方法:通过预定义的句式模板或受限语法结构来规范需求描述,降低自然语言的歧义性,并在此基础上引入不同程度的形式化语义支持,以实现需求到逻辑规约的转化.代表性方法如 EARS(Easy Approach to Requirements Syntax)[30],通过 Ubiquitous、Event-Driven、State-Driven 等 5 种模板将自然语言需求结构化; FRETish[31]提供五字段模板供填写结构化需求,然后自动将其翻译为时序逻辑公式 (支持 LTL/MTL); SpeAR[32]采用基于模式的方法——用户从 Dwyer 规范模式(absence、existence、precedence、response 等)中选择模板,手动转化为 Lustre 形式化规约;Hanfor[33]让用户通过选择 Scope(如 Globally)和 Pattern(如“if R holds then S holds after at most T time units”)并填空完成形式化.其优势在于简单易用、学习成本低,但它们仅提供句式规范,缺乏资源抽象机制,未显式解绑需求与具体设备描述;仅能做到需求句式模板级的复用.

基于本体的方法:这类方法引入领域本体作为需求描述的语义基础,结合模板化语法降低歧义.代表性工作包括 ReSA[34],它面向汽车嵌入式系统,基于汽车系统本体(采用描述逻辑编码)为需求提供类型约束和语法公理,并与 EAST-ADL 架构集成,支持需求在多抽象层级上的描述与精化.用户通过固定句式(Boilerplate)模板填写领域概念完成需求编写(如“if driver selects ASL speed control then ASL shall be enabled”),工具将其转化为布尔逻辑表达式.

基于形式化建模的方法:以精确的数学模型描述系统行为,通过显式定义系统输入与输出之间的逻辑关系来刻画需求.代表性工作为 SCR(Software Cost Reduction)方法[22-24],其核心建模概念为“监视变量”(monitored variables,表征系统输入)与“控制变量”(controlled variables,表征系统输出),通过定义两者之间的逻辑依赖关系(如条件触发表、事件表等)精确描述系统行为. 其复用粒度局限于组件级,即基于四变量模型分解出的独立规约模块可被单独验证和组合复用,但由于缺乏对物理设备的中间抽象层,这些组件仍与具体硬件变量紧密耦合,跨项目迁移时需重新适配,难以支持更高层级的复用.

基于混合表达的方法:这类方法不局限于单一表达范式,而是同时提供多种需求描述机制,允许用户根据

场景灵活选择或组合使用自然语言、模板化句式、形式化约束等不同精度的表达方式.SysML v2[35]是该类方法的典型代表,它将需求作为一等模型元素,采用三种需求表达机制:通过 doc 注释承载自然语言描述;通过受限需求语句使用预定义关键字和模板句式规范需求表达,在自然语言与形式化之间取得平衡;通过需要的约束定义形式化约束,支持精确的逻辑表达式自动验证.SysML v2 还支持需求模板的参数化复用.

现有系统需求描述方法分别通过语法约束、形式化验证和本体推理等来降低需求的歧义性.但它们忽略了对物理或设备资源抽象的关注,不支持需求描述与硬件细节的解绑定;需求复用粒度主要是模板、模式、组件或模型库级复用;也缺少对从系统需求到软件需求映射的支持.

6.3 工作比较

我们选取五类具有代表性的方法/语言进行对比,从九个维度展开：领域（方法适用的应用领域）、需求层次（描述对象是系统层还是软件层）、核心机制（降低歧义或表达需求的主要手段）、资源抽象（是否支持及抽象能力强弱）、软硬件解绑（需求与具体设备描述的绑定程度）、原子操作定义（是否支持细粒度的可组合操作单元）、输出产物（方法最终生成的结果形式）、自动转换支持（从系统需求到软件规约的自动化程度）以及复用粒度（需求复用的基本单元）.具体对比如表 13 所示.

表 13 SpaceRDL 与代表性需求描述方法的对比

维度	EARS	SCR	SPARDL	ReSA	SysML v2.0	**SpaceRDL**
领域	通用	嵌入式	航天	汽车	通用	**航天**
需求层次	系统层	系统+软件层	软件层	系统层	系统层	**系统层**
核心机制	句式模板	变量间关系	模式+周期任务	本体+固定句式	自然语言+模板+形式化表达	**资源库+操作语句**
资源抽象	无	变量抽象	弱（数据字典）	有（汽车本体）	中（接口建模）	**强（航天资源库）**
软硬件解绑	弱	无	无	弱	弱	**强**
原子操作定义	无	无	有（模式/周期）	无	无	**有（操作算子体系）**
输出产物	结构化需求	需求文本	可执行模型	需求文本	可执行模型	**软件需求规约**
自动转换支持	无	无	弱（原型生成）	无	弱（依赖工具）	**强（语言内嵌）**
复用粒度	模板级	组件级	模式级	模板级	模型库级	**原子操作指令**

可以看出,SpaceRDL 在多个维度上与已有工作形成区分.与 EARS 相比,SpaceRDL 不再局限于句式规范或逻辑验证,而是通过资源库机制和操作语句实现系统需求的精确描述.与 SPARDL 相比,SpaceRDL 跨越系统需求和软件,以系统任务意图为起点，通过资源库将设备能力与具体型号设备细节解绑,系统需求描述聚焦于“做什么”而非“用什么做”,同时通过操作指令的内涵定义建立系统需求到软件需求的映射,基于映射规则推导出软件需求.相比 ReSA,SpaceRDL 的资源库独立于特定架构设计,支持原子操作需求的复用,和从系统需求到软件需求的映射.相比 SysML v2 和 SCR,SpaceRDL 建立显式的软硬件解耦支持原子操作需求复用,内嵌映射规则实现从系统需求到软件需求的映射.综上，SpaceRDL 是在对航天控制系统特征理解的基础上,针对系统资源库构建、原子操作指令设计及内嵌映射机制的系统性设计,为航天控制系统需求工程提供了新的解决方案.

7 总结与展望

本文提出了系统级需求描述语言 SpaceRDL,其核心创新在于引入“系统资源”作为统一的抽象层,将物理设备,逻辑算法与数据单元封装为可配置的资源对象,并构建了系统资源库体系.SpaceRDL 以面向资源的操作指令为核心,通过结构化的语法和原子操作指令设计,使工程师能够在控制逻辑层面精确定义对系统资源的操作;同时通过内嵌的映射机制,完成从任务意图到软件需求的自动化推导,为构建新一代航天控制系统软件工程奠定了坚实的基础.

通过太阳搜索控制系统案例,实证了 SpaceRDL 的工程可行性与有效性.案例表明,基于 SpaceRDL 的需求描述不仅能够清晰、无歧义地定义复杂的多模式控制逻辑, 更能通过映射规则,自动生成精确的软件需求规约.这改变了传统依赖人工解读和转换的需求工程模式,显著提升了需求的精确性、一致性以及开发效率.更重要的是,通过资源模板与需求模式的分离机制,SpaceRDL 实现了需求与硬件细节的解绑,使得核心需求资产能够在不同型号或项目间高效复用.

尽管 SpaceRDL 在提升需求描述质量与复用性方面展现出显著优势,但其发展与完善仍是一个持续的过程.未来,我们将从以下几个方向开展进一步的工作:(1) 丰富语义体系,增强对安全性、可靠性等质量属性的表达能力;(2) 融合形式化验证方法,实现需求规约的一致性、完整性及安全属性的自动验证;(3) 引入大语言模型,探索从自然语言需求到 SpaceRDL 规约的自动生成;(4) 研究上下文感知的需求复用推荐机制;(5)探索历史型号文档中系统资源的自动提取与构建方法.

References

[1] Yuan L, Jiang TT, Wei CL, Yang MF. Advances and perspectives of space control technology. Acta Automatica Sinica, 2023,49(3):476–493 (in Chinese). [doi: 10.16383/j.aas.c220792]

[2] Gu B, Dong YW, Wang Z. Formal Modeling Approach for Aerospace Embedded Software. Ruan Jian Xue Bao/ Journal of Software, 2015, 26(2): 321-331(in Chinese). [doi: 10.13328/j.cnki.jos.004784]

[3] Yang MF, Gu B, Duan ZH, Jin Z, Zhan NJ, Dong YW, Tian C, Li G, Dong XG, Li XF. Intelligent program synthesis framework and key scientific problems for embedded software. Chinese Space Science and Technology, 2022, 42(4): 1–7 (in Chinese). [doi: 10.16708/j.cnki.1000-758X.2022.0046]

[4] Broy M, Stauner T. Requirements engineering for embedded systems. 1999.

[5] Naumchev A, Meyer B, Mazzara M,Galinier F, Bruel JM, Ebersold S. Autoreq: Expressing and verifying embedded software requirements. arXiv:1710.02801v1, 2017. [doi: 10.48550/arXiv.1710.02801]

[6] Zuo WJ, Wang XL, Huang C,Dong Y. Analysis and Practice of Implicit Requirement for Aerospace Embedded Software. Measurement & Control Technology, 2023, 42(10): 24-29 (in Chinese). [doi: 10.19708/j.ckjs.2023.10.004]

[7] Luo Y, Wang Y, Li X, Gu B, Jin Z. Leveraging Large Language Models for Reusable Requirements Management in Aerospace Software. In: 2025 IEEE 33rd International Requirements Engineering Conference (RE 2025). IEEE, 2025. 361-365. [doi: 10.1109/RE63999.2025.00042]

[8] Luo Y, Liu Y, Li X, Dong X, Gu B, Jin Z, Yang M. Taxonomy-Guided Reasoning for Requirements Classification: A Study in Aerospace Industry. In: 2025 IEEE 33rd International Requirements Engineering Conference (RE 2025). IEEE, 2025. 332-343. [doi: 10.1109/RE63999.2025.00039]

[9] Lin J, Luo Y, Chen X, Gu B, Jin Z. Automatic Generation of Structured Requirements for Aerospace Embedded Systems Using LLMs. In: 2025 IEEE International Requirements Engineering Conference Workshops (REW 2025). IEEE, 2025. 181-188. [doi: 10.1109/REW66121.2025.00028]

[10] Harel D. Statecharts: a visual formalism for complex systems. Science of Computer Programming, 1987, 8(3): 231-274. [doi: 10.1016/0167-6423(87)90035-9]

[11] The MathWorks. Stateflow® and Stateflow® Coder™ User's Guide[M]. Natick, MA: The MathWorks, 2009.

[12] Rockstrom A, Saracco R. SDL - CCITT Specification and Description Language. IEEE Transactions on Communications, 1982, 30(6): 1310-1318. [doi: 10.1109/TCOM.1982.1095599]

[13] Jahanian F, Mok AK. Modechart: a specification language for real-time systems. IEEE Transactions on Software Engineering, 1994, 20(12): 933-947. [doi: 10.1109/32.368134]

[14] Wang Z, Li J, Zhao Y, Qi Y, Pu G, He J, Gu B. SPARDL: A Requirement Modeling Language for Periodic Control System. In: Margaria T, Steffen B, eds. Leveraging Applications of Formal Methods, Verification, and Validation. International Symposium on Leveraging Applications of Formal Methods (ISoLA 2010). Berlin: Springer, 2010. 628-642. [doi: 10.1007/978-3-642-16558-0_48]

[15] Henzinger TA, Horowitz B, Kirsch CM. Giotto: A Time-Triggered Language for Embedded Programming. In: Proc. of the 1st International Workshop on Embedded Software (EMSOFT 2001). Tahoe City: Springer, 2001. [doi: 10.1007/3-540-45449-7_12]

[16] Tsai JJP. A Knowledge-Based System for Software Design. IEEE Journal on Selected Areas in Communications, 1988, 6(5): 828-841. [doi: 10.1109/49.634]

[17] Tsai JJP, LIU AL. Experience on Knowledge-Based Software Engineering: A Logic-Based Requirements Language and Its Industrial Applications. Journal of Systems and Software, 2009, 82(10): 1578-1587. [doi: 10.1016/j.jss.2009.03.019]

[18] Tsai JJP, JANG HC. A Knowledge-Based Approach for the Specification and Analysis of Real-Time Software Systems. International Journal on Artificial Intelligence Tools, 1992, 1(1): 1-35.[doi: 10.1142/9789814354714_0016]

[19] Auernheimer B, Kemmerer RA. ASLAN User's Manual. Santa Barbara: University of California, 1985.

[20] Auernheimer B, Kemmerer RA. RT-ASLAN: A Specification Language for Real-Time Systems. IEEE Transactions on Software Engineering, 1986, SE-12(9): 879-889. [doi: 10.1109/TSE.1986.6313044]

[21] Ghezzi C, Kemmerer RA. ASTRAL: An assertion language for specifying realtime systems. In: Proc. of the 3rd European Software Engineering Conf. Milan: Springer, 1991. 122–146. [doi: 10.1007/3540547428_46]

[22] Heninger KL. Specifying Software Requirements for Complex Systems: New Techniques and Their Application. IEEE Transactions on Software Engineering, 1980, SE-6(1): 2-13. [doi: 10.1109/TSE.1980.230208]

[23] Parnas DL, Madey J. Functional Documents for Computer Systems. Science of Computer Programming, 1995, 25(1): 41-61. [doi: 10.1016/0167-6423(95)96871-J]

[24] Heitmeyer CL. Software Cost Reduction. MARCINIAK J J. Encyclopedia of Software Engineering. Hoboken: John Wiley & Sons, 2002. [doi: 10.1002/0471028959.sof307]

[25] Chen XH, Liu SB, Jin Z. Survey on Requirements Description of Embedded System. Ruan Jian Xue Bao/Journal of Software, 2025, 36(1): 27－46 (in Chinese). [doi: 10.13328/j.cnki.jos.007157]

[26] Jackson M. Problem Frames: Analyzing and Structuring Software Development Problems. Addison-Wesley, 2001.

[27] Jackson M, The meaning of requirements, Annals of Software Engineering 3 (1997) 5–21. [doi: 10.1023/A:1018990005598]

[28] Jin Z, Environment Modelling-Based Requirements Engineering for Software Intensive Systems. Amsterdam: Elsevier, 2018. [doi: 10.1016/C2014-0-00030-5]

[29] Wang X, Chen X, Yang X, Yang B. Requirements Patterns for Complex Embedded Systems. In: Proceedings of 2022 IEEE 30th International Requirements Engineering Conference Workshops (REW 2022). Melbourne: IEEE, 2022. 14-17. [doi: 10.1109/REW56159.2022.00011]

[30] Mavin A, Wilkinson P, Harwood A, Novak M. Easy approach to requirements syntax (EARS). In: Proc. of the 17th IEEE Int'l Requirements Engineering Conf. Atlanta: IEEE, 2009. 317–322. [doi: 10.1109/RE.2009.9]

[31] Farrell M, Luckcuck M, Sheridan O, Monahan R. Towards Refactoring FRETish Requirements. NFM 2022: 272-279. [doi: 10.48550/arXiv.2201.04531]

[32] Fifarek AW, Wagner LG, Hoffman JA, Rodes BD, Aiello MA, Davis JA. SpeAR v2.0: Formalized Past LTL Specification and Analysis of Requirements. In: NASA Formal Methods Symposium (NFM 2017). Cham: Springer, 2017. 420-426. [doi: 10.1007/978-3-319-57288-8_30]

[33] Hauff N, Henkel E, Kolzer T, Langenfeld V, Podelski A. Hanfor: Requirements Formalisation and Beyond. REFSQ Workshops 2025.

[34] Mahmud N, Seceleanu C, Ljungkrantz O. ReSA: An ontology-based requirement specification language tailored to automotive systems. SIES 2015: 1-10. [doi: 10.1109/SIES.2015.7185035]

[35] Object Management Group. OMG Systems Modeling Language (SysML) Version 2.0 Beta Specification. 2023. Available: https://www.omg.org/spec/SysML/2.0/Beta1.

附中文参考文献

[1] 袁利, 姜甜甜, 魏春岭, 杨孟飞.空间控制技术发展与展望. 自动化学报,2023,49(3):476-493., 2015, 26(2):321-331. [doi: 10.16383/j.aas.c220792]

[2] 顾斌, 董云卫, 王政. 面向航天嵌入式软件的形式化建模方法.软件学报, 2015, 26(2): 321-331. [doi: 10.13328/j.cnki.jos.004784]

[3] 杨孟飞, 顾斌, 段振华, 等. 嵌入式软件智能合成框架及关键科学问题. 中国空间科学技术, 2022:1-6. [doi: 10.16708/j.cnki.1000-758X.2022.0046]

[6] 左万娟, 王小丽, 黄晨, 董燕. 航天嵌入式软件隐含需求分析与实践. 测控技术, 2023, 42(10): 24-29. [doi: 10.19708/j.ckjs.2023.10.004]

[8] 王小齐, 陈小红, 金芝, 顾斌, 綦艳霞.基于投影的嵌入式系统需求分析方法.软件学报, 2024,35(12):5397-5418. [doi: 10.13328/j.cnki.jos.007081]

[25] 陈小红, 刘少彬, 金芝, 嵌入式系统的需求描述综述, 软件学报, 2025, 36(1): 27-46. [doi: 10.13328/j.cnki.jos.007157]



陈小红(1982－),女,博士,教授,博士生导师,主要研究领域为需求工程、知识工程和形式化方法.



綦艳霞(1973－),女,博士,研究员,主要研究领域为可信软件、嵌入式系统与计算机控制.



阮锟(2001－),女,硕士,主要研究领域为需求工程.



林靖凯(2003－),男,硕士生,主要研究领域为需求工程.



金芝(1962－),女,博士,教授,主要研究领域为软件需求工程,智能化软件工程与知识工程.



钟睿明(1985－),男,博士,高级工程师,主要研究领域为可信软件、嵌入式软件、计算机控制.



顾斌(1968－),男,博士,研究员,主要研究领域为可信软件、嵌入式系统与计算机控制.



杨孟飞(1962-),男, 博士,研究员, 博士生导师,CCF 高级会员,主要研究领域为空间飞行器系统总体,控制系统,控制计算机和可信软件.